

#SystemDesign

"Hệ thống tuyệt vời được xây dựng,
chứ không phải đoán mò."

Ghi chú KIẾN TRÚC MÁY TÍNH

Học • Hiểu • Ứng dụng



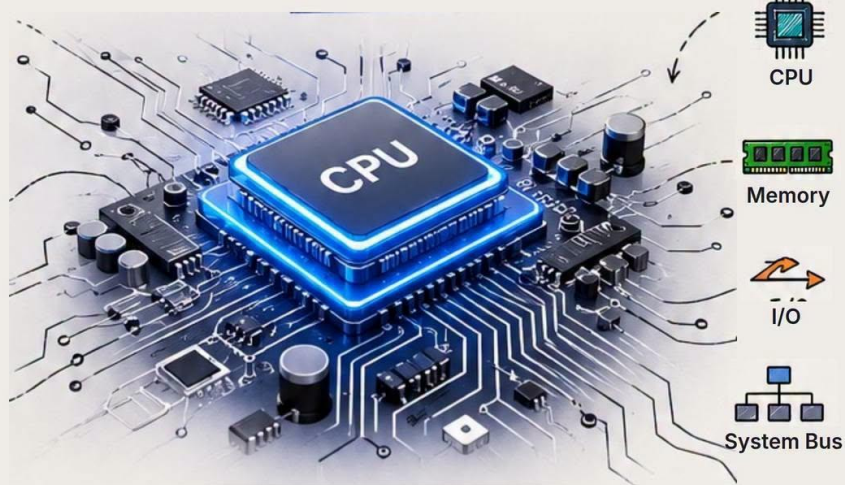
Thiết kế
thông minh



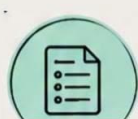
Hiểu
sâu sắc



Xây dựng
tự tin



Kiến trúc là thứ vô hình,
nhưng tác động của nó ở khắp mọi nơi.



Đầu vào



Xử lý
(CPU)



Bộ nhớ



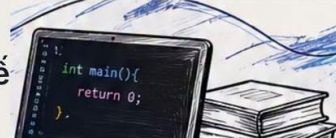
Lưu trữ



Đầu ra

- ✓ Khái niệm rõ ràng
- ✓ Cấu trúc mạch lạc
- ✓ Sẵn sàng thi cử & phỏng vấn

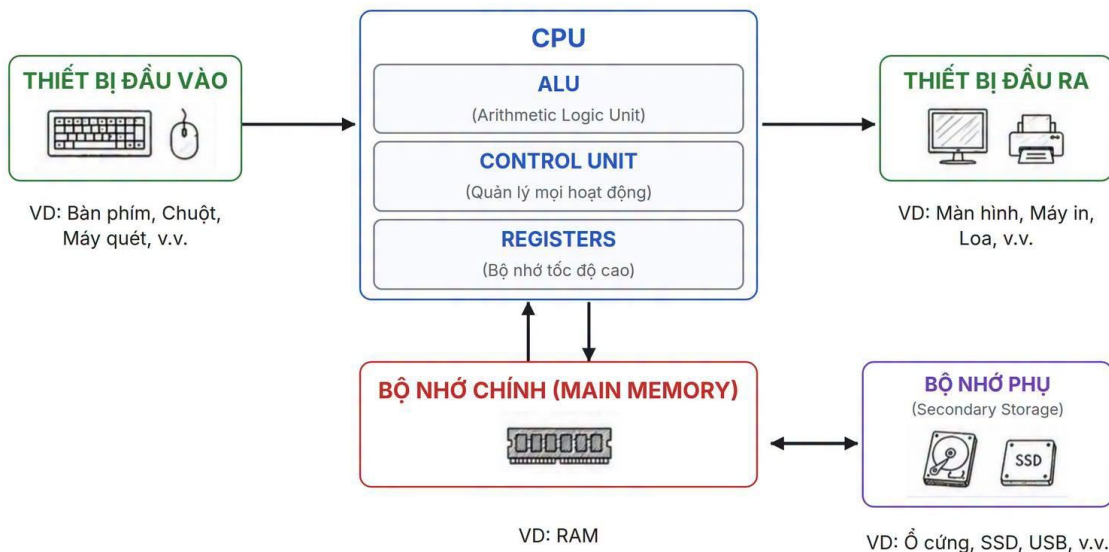
Tư duy. Thiết kế
Tối ưu.



≡ KIẾN TRÚC MÁY TÍNH ≡

CHƯƠNG 1 : GIỚI THIỆU

Kiến trúc Máy tính là nền tảng của một hệ thống máy tính. Nó đề cập đến **thiết kế, cấu trúc và tổ chức** của máy tính cùng cách các thành phần **phối hợp với nhau** để thực hiện tác vụ một cách hiệu quả.



CHÚNG TA SẼ HỌC GÌ

- ✓ CPU, ALU, Control Unit và Registers
- ✓ Tổ chức Memory và Storage
- ✓ System Bus và cách giao tiếp
- ✓ Instruction Cycle và thực thi lệnh
- ✓ Hiệu năng, Pipelining & Parallel Processing
- ✓ Các kiến trúc hiện đại và nhiều hơn nữa

ĐIỂM CHÍNH

- ⚡ Thiết kế của một hệ thống máy tính
- ⚡ Dữ liệu được xử lý bên trong như thế nào
- ⚡ Lệnh được thực thi ra sao
- ⚡ Các thành phần giao tiếp với nhau thế nào
- ⚡ Hiệu năng được cải thiện bằng cách nào



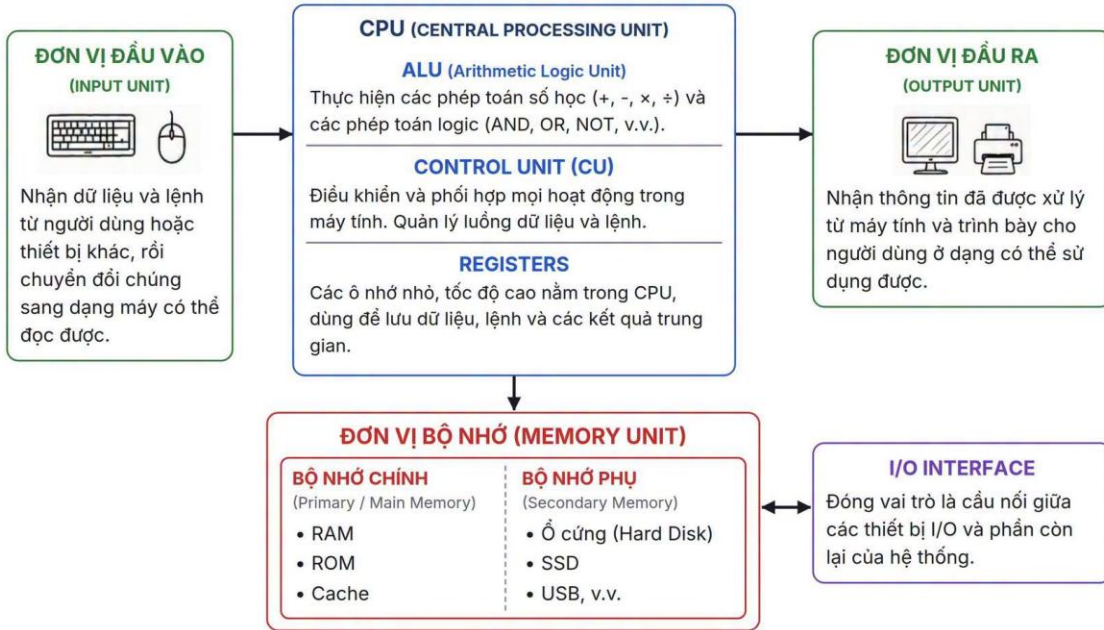
TỔNG KẾT

Kiến trúc Máy tính giúp chúng ta hiểu máy tính hoạt động bên trong như thế nào và mọi bộ phận phối hợp với nhau ra sao để hoàn thành bất kỳ tác vụ nào một cách chính xác và hiệu quả.



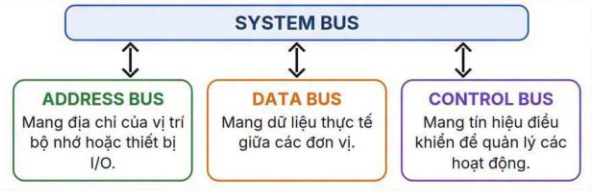
2. TỔ CHỨC CƠ BẢN CỦA MÁY TÍNH

Máy tính được tổ chức thành các đơn vị chức năng chính, phối hợp với nhau để **nhận dữ liệu, xử lý, lưu trữ** và **tạo ra kết quả đầu ra**.



SYSTEM BUS

System bus là đường truyền thông tin dùng để chuyển dữ liệu, địa chỉ và tín hiệu điều khiển giữa CPU, bộ nhớ và các thiết bị I/O.



CÁCH HOẠT ĐỘNG

- 1 Input unit nhận dữ liệu / lệnh.
- 2 Dữ liệu được lưu vào bộ nhớ.
- 3 CPU nạp (fetch) lệnh từ bộ nhớ.
- 4 CPU giải mã và thực thi các lệnh.
- 5 Kết quả được lưu vào bộ nhớ hoặc gửi đến output unit.
- 6 Output unit trình bày kết quả cuối cùng.

ĐIỂM CHÍNH

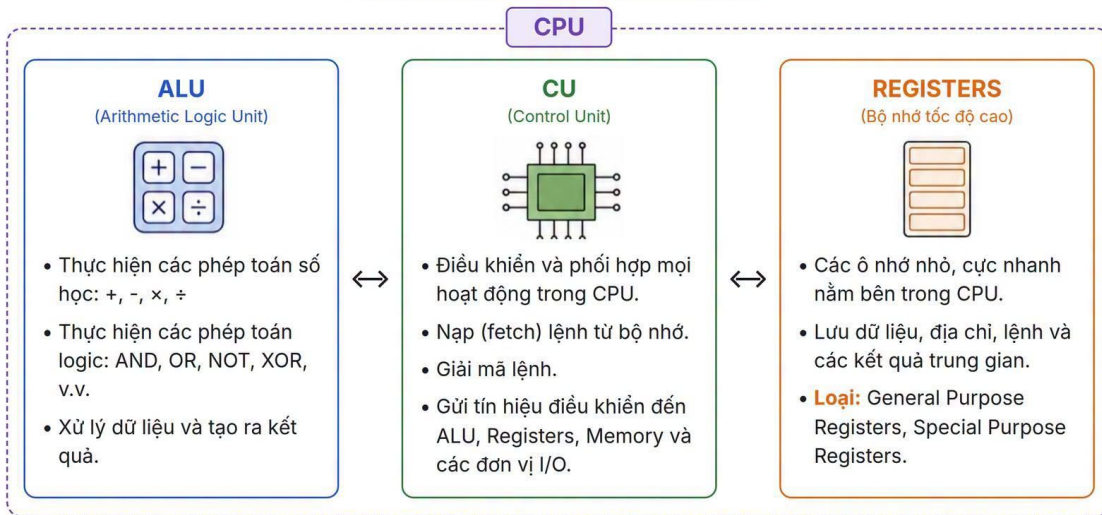
- 🔥 Mọi đơn vị được kết nối với nhau và hoạt động như một hệ thống thống nhất.
- 🔥 CPU là bộ não của máy tính.
- 🔥 Bộ nhớ lưu trữ dữ liệu, lệnh và kết quả.
- 🔥 Các đơn vị I/O cho phép giao tiếp với thế giới bên ngoài.
- 🔥 System bus đảm bảo dữ liệu được truyền trơn tru giữa mọi đơn vị.

3. CPU, ALU, CU & REGISTERS

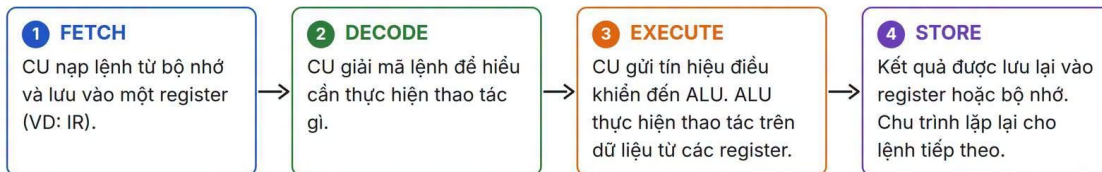
CPU (Central Processing Unit) là bộ não của máy tính.

Nó thực hiện toàn bộ quá trình xử lý và điều khiển hoạt động của mọi đơn vị.

CPU = ALU + CU + REGISTERS



CÁCH CHÚNG PHỐI HỢP VỚI NHAU



CÁC LOẠI REGISTER

- General Purpose Registers** - Dùng để lưu dữ liệu tạm thời trong quá trình xử lý.
- Special Purpose Registers** - Có các chức năng cụ thể trong CPU.
 - PC (Program Counter)** - Giữ địa chỉ của lệnh tiếp theo.
 - IR (Instruction Register)** - Giữ lệnh hiện tại.
 - MAR (Memory Address Register)** - Giữ địa chỉ của vị trí bộ nhớ.
 - MDR (Memory Data Register)** - Giữ dữ liệu đọc từ hoặc ghi vào bộ nhớ.

ĐIỂM CHÍNH

- ALU thực hiện các phép tính và phép toán logic.
- CU điều khiển và phối hợp mọi hoạt động.
- Registers cung cấp bộ nhớ tốc độ cao bên trong CPU.
- Ba đơn vị này phối hợp với nhau để thực thi lệnh một cách hiệu quả.
- CPU là trái tim của hệ thống máy tính.

≡ 4. INSTRUCTION CYCLE ≡

Instruction Cycle (chu trình lệnh) là quá trình CPU **nạp (fetch)** một lệnh từ bộ nhớ, **giải mã (decode), thực thi (execute)** và **lưu (store)** kết quả. Chu trình này lặp lại cho mọi lệnh trong chương trình.

CÁC BƯỚC TRONG INSTRUCTION CYCLE

1 FETCH

Memory → IR

- CPU dùng địa chỉ trong Program Counter (PC) để nạp lệnh tiếp theo từ bộ nhớ.
- Lệnh được nạp vào Instruction Register (IR).
- PC được cập nhật để trở đến lệnh tiếp theo.

2 DECODE



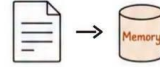
- Control Unit (CU) giải mã lệnh nằm trong IR.
- Xác định thao tác cần thực hiện và các toán hạng (dữ liệu) cần dùng.
- CU tạo tín hiệu điều khiển cho bước tiếp theo.

3 EXECUTE



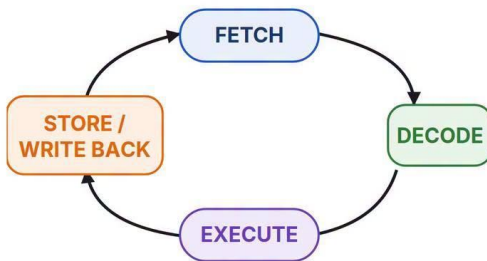
- ALU thực hiện thao tác cần thiết (số học / logic).
- Toán hạng được lấy từ các register hoặc bộ nhớ.
- Kết quả được lưu vào register hoặc bộ nhớ.

4 STORE / WRITE BACK



- Kết quả thực thi được ghi lại vào register đích hoặc bộ nhớ.
- Các cờ trạng thái (Zero, Carry, Sign, v.v.) cũng có thể được cập nhật.
- Chu trình lặp lại cho lệnh tiếp theo.

LUỒNG CỦA INSTRUCTION CYCLE



ĐIỂM QUAN TRỌNG

- 🔥 Số chu kỳ xung nhịp (clock cycle) cần cho một lệnh phụ thuộc vào loại lệnh và hệ thống.
- 🔥 Một số lệnh có thể cần thêm chu kỳ để nạp toán hạng từ bộ nhớ.
- 🔥 Instruction cycle là thao tác cơ bản mà CPU thực hiện.
- 🔥 Thực thi nhanh hơn phụ thuộc vào việc giảm thời gian ở mỗi giai đoạn.

VAI TRÒ CỦA CÁC THÀNH PHẦN CPU TRONG INSTRUCTION CYCLE

Thành phần	Vai trò trong Instruction Cycle	Ví dụ
Program Counter (PC)	Giữ địa chỉ của lệnh tiếp theo	PC = 2050H
Instruction Register (IR)	Giữ lệnh hiện tại	IR = 8 3 4 1 (lệnh ví dụ)
Control Unit (CU)	Giải mã lệnh và phát tín hiệu điều khiển	Tạo tín hiệu cho ALU, Memory, I/O
ALU	Thực thi thao tác	Thực hiện ADD, SUB, AND, OR, v.v.
Registers	Lưu dữ liệu, địa chỉ, kết quả trung gian	Acc, R1, R2, v.v.
Memory	Lưu lệnh và dữ liệu	Lưu chương trình và các biến



TỔNG KẾT

Mọi lệnh đều đi qua bốn giai đoạn Fetch - Decode - Execute - Store. CPU lặp đi lặp lại chu trình này hàng tỷ lần mỗi giây, và tốc độ của máy tính phụ thuộc vào việc rút ngắn thời gian của từng giai đoạn.

5. ĐỊNH DẠNG & CÁC LOẠI INSTRUCTION



Instruction (lệnh) là một mệnh lệnh được đưa cho CPU để thực hiện một **thao tác cụ thể**. Nó được lưu trong bộ nhớ và gồm hai phần chính: **OPCODE** (mã thao tác) và **OPERAND** (dữ liệu hoặc địa chỉ).

INSTRUCTION FORMAT

Cách bố trí các bit trong một lệnh được gọi là **format** (định dạng) của lệnh. Một định dạng lệnh tổng quát như sau:

OPCODE (Mã thao tác)	ADDRESS / MODE (Nếu cần)	OPERAND / DATA (Nếu cần)
Tổng độ dài lệnh (tính bằng bit)		

OPCODE

Chỉ định thao tác cần được thực hiện.

OPERAND

Dữ liệu, địa chỉ hoặc register mà thao tác được thực hiện trên đó.

CÁC LOẠI LỆNH

Lệnh được phân loại dựa trên thao tác mà chúng thực hiện.

Loại	Mô tả	Ví dụ	Ý nghĩa ví dụ
1. Data Transfer	Chuyển dữ liệu giữa các register, bộ nhớ và I/O.	MOV R1, R2	Sao chép dữ liệu từ R2 sang R1.
2. Arithmetic	Thực hiện các phép toán số học.	ADD R1, R2	$R1 \leftarrow R1 + R2$
3. Logical	Thực hiện các phép toán logic (AND, OR, NOT, XOR, v.v.).	AND R1, R2	$R1 \leftarrow R1 \text{ AND } R2$
4. Control Transfer	Thay đổi trình tự thực thi.	JMP 2000H	Nhảy đến vị trí bộ nhớ 2000H.
5. Input / Output	Chuyển dữ liệu giữa CPU và các thiết bị I/O.	IN R1, 80H	Đọc dữ liệu từ cổng I/O 80H vào R1.
6. Stack	Thực hiện các thao tác trên stack (push, pop).	PUSH R1	Đẩy nội dung của R1 vào stack.
7. Machine Control	Điều khiển ngắt (interrupt), các cờ hệ thống, v.v.	HLT	Dừng việc thực thi chương trình.

CÁC ĐỊNH DẠNG LỆNH PHỔ BIẾN

Loại định dạng	Cấu trúc	Dùng cho
1. Three Address	OPCODE A1 A2 A3	Thao tác phức tạp, nhiều toán hạng
2. Two Address	OPCODE A1 A2	Hầu hết các lệnh đa dụng
3. One Address	OPCODE A	Thao tác dựa trên accumulator
4. Zero Address	OPCODE	Thao tác dựa trên stack

CÁC ADDRESSING MODE (TÓM TẮT)

Mode	Ý nghĩa	Ví dụ
Immediate	Toán hạng là một phần của lệnh	MOV R1, #25
Direct	Địa chỉ của toán hạng được cho trực tiếp	MOV R1, 2050H
Register	Toán hạng nằm trong một register	MOV R1, R2
Register Indirect	Địa chỉ nằm trong một register	MOV R1, (R2)
Indexed	Địa chỉ hiệu dụng = Base + Index	MOV R1, 1000H(R2)

6. ADDRESSING MODES

Addressing Mode là phương pháp mà một lệnh sử dụng để chỉ định vị trí của toán hạng (operand).



Nó cho CPU biết cách tìm dữ liệu thực tế hoặc **effective address** (địa chỉ hiệu dụng) cần cho một thao tác.

CÁC ADDRESSING MODE PHỔ BIẾN

Mode	Mô tả	Cách hoạt động	Ví dụ	Effective Address (EA)
1. Immediate #25	Toán hạng là một phần của chính lệnh đó.	Giá trị được dùng trực tiếp làm toán hạng.	MOV R1, #25 (Nạp 25 vào R1)	Không cần (Giá trị nằm trong lệnh)
2. Direct 2050H	Trường địa chỉ chứa địa chỉ bộ nhớ của toán hạng.	Địa chỉ trong lệnh trỏ thẳng đến vị trí bộ nhớ.	MOV R1, 2050H	EA = 2050H
3. Register R2	Toán hạng nằm trong một register.	Lệnh chỉ định tên của register.	MOV R1, R2	Không cần (Dữ liệu nằm trong register)
4. Register Indirect R2 → 5000H	Register chứa địa chỉ của toán hạng.	Địa chỉ trong register trỏ đến vị trí bộ nhớ của toán hạng.	MOV R1, (R2)	EA = (R2) (Nội dung của R2)
5. Indexed 2000H + R3	Địa chỉ cơ sở được cộng với nội dung của index register.	EA có được bằng cách cộng nội dung index register vào trường địa chỉ.	MOV R1, 2000H(R3)	EA = 2000H + (R3)
6. Base Register R4 + 1000H	Nội dung của base register được cộng vào trường địa chỉ.	EA có được bằng cách cộng nội dung base register vào trường địa chỉ.	MOV R1, 1000H(R4)	EA = (R4) + 1000H
7. Relative (PC Relative) PC + 20	Địa chỉ có được bằng cách cộng một độ dịch (displacement) vào Program Counter.	Dùng trong các lệnh rẽ nhánh / nhảy (relative addressing).	JMP 20 (Nhảy tới trước 20)	EA = (PC) + 20
8. Auto Increment R1 → (R1)+	Register trỏ đến toán hạng; sau khi truy cập, register được tăng lên.	Sau khi lấy toán hạng từ bộ nhớ, register được tăng giá trị.	MOV R0, (R1)+	EA = (R1) Sau đó R1 = R1 + d
9. Auto Decrement -(R1) → R1	Register được giảm trước, rồi mới dùng để truy cập toán hạng.	Register được giảm trước, sau đó vị trí bộ nhớ được truy cập.	MOV R0, -(R1)	R1 = R1 - d EA = (R1)



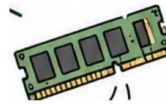
GHI CHÚ THÊM

- EA (Effective Address) là vị trí bộ nhớ thực tế nơi toán hạng được tìm thấy.
- Các addressing mode khác nhau mang lại sự linh hoạt và giúp giảm kích thước lệnh.
- Chọn đúng addressing mode có thể cải thiện hiệu năng và hiệu quả của mã lệnh.

SƠ SÁNH NHANH

- Immediate → Dữ liệu nằm trong lệnh
- Direct → Địa chỉ nằm trong lệnh
- Register → Dữ liệu nằm trong register
- Register Indirect → Địa chỉ nằm trong register
- Indexed / Base → Base + Index
- Relative → Dựa trên PC
- Auto Inc / Dec → Register tự cập nhật khi truy cập

7. TỔ CHỨC BỘ NHỚ

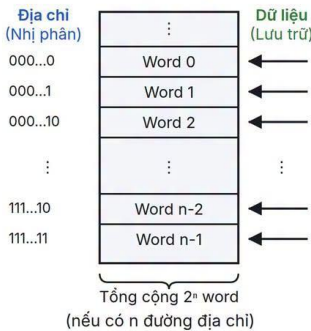


Tổ chức bộ nhớ (memory organization) đề cập đến cách bộ nhớ chính của máy tính được **cấu trúc**, **đánh địa chỉ** và **vận hành** để lưu trữ lệnh và dữ liệu một cách hiệu quả.

1. KHÁI NIỆM CƠ BẢN

- Bộ nhớ chính là một mảng lớn gồm các ô lưu trữ.
- Mỗi ô (word) có một địa chỉ duy nhất.
- CPU có thể đọc từ hoặc ghi vào bất kỳ vị trí bộ nhớ nào.
- Bộ nhớ được đo bằng byte. (1 Byte = 8 bit)

Bộ nhớ như một mảng các Word



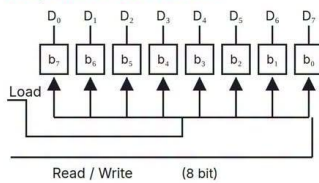
THUẬT NGỮ CHÍNH

- Word:** Đơn vị dữ liệu cơ bản được truyền giữa CPU và bộ nhớ.
- Address:** Số nhị phân duy nhất xác định một vị trí bộ nhớ.
- Capacity:** Tổng số byte có thể lưu trữ.
- Access Time:** Thời gian cần để đọc hoặc ghi dữ liệu.
- Memory Cycle Time:** Thời gian tối thiểu giữa hai lần truy cập bộ nhớ liên tiếp.

2. MEMORY UNIT

Một memory unit gồm một nhóm flip-flop (hoặc latch) dùng để lưu trữ một word.

Ví dụ: memory unit 8-bit



3. ĐÁNH ĐỊA CHỈ BỘ NHỚ

Nếu bộ nhớ có 2^n vị trí, nó cần n đường địa chỉ.

Ví dụ:

Nếu bộ nhớ có 1024 word (2^{10}), thì cần 10 đường địa chỉ (A0 đến A9).

Số word	Số đường địa chỉ (n)
16 (2^4)	4
256 (2^8)	8
1024 (2^{10})	10
4096 (2^{12})	12

4. CÁC LOẠI BỘ NHỚ

1. RAM (Random Access Memory)

- Bộ nhớ đọc / ghi.
- Khả biến (mất dữ liệu khi tắt nguồn).
- Ví dụ: DRAM, SRAM.

2. ROM (Read Only Memory)

- Bộ nhớ chỉ đọc.
- Bất biến (dữ liệu được giữ lại).
- Ví dụ: PROM, EPROM, EEPROM.

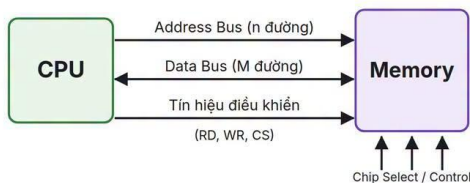
3. Cache Memory

- Bộ nhớ nhỏ, nhanh, nằm giữa CPU và bộ nhớ chính.
- Lưu dữ liệu được dùng thường xuyên.

5. THAO TÁC BỘ NHỚ

Thao tác	Chức năng	Tín hiệu điều khiển	Mô tả
Read	Đọc dữ liệu từ bộ nhớ	RD = 0, WR = 1	Địa chỉ được đặt lên các đường địa chỉ. Dữ liệu được đọc từ bộ nhớ ra các đường dữ liệu.
Write	Ghi dữ liệu vào bộ nhớ	RD = 1, WR = 0	Địa chỉ được đặt lên các đường địa chỉ. Dữ liệu từ các đường dữ liệu được ghi vào bộ nhớ.
Idle	Không thao tác	RD = 1, WR = 1	Bộ nhớ ở trạng thái không hoạt động.

6. MEMORY INTERFACE



Address Decode Logic

Giải mã địa chỉ và chọn vị trí bộ nhớ cần thiết (chip select).

ĐIỂM QUAN TRỌNG

- Bộ nhớ được tổ chức như một mảng các word.
- Các đường địa chỉ xác định vị trí nào đang được truy cập.
- Các đường dữ liệu truyền dữ liệu thực tế.
- Tín hiệu điều khiển quyết định thao tác là Read hay Write.
- Bộ nhớ nhanh hơn với thời gian truy cập thấp hơn giúp cải thiện hiệu năng hệ thống.

8. BỘ NHỚ CACHE & PHÂN CẤP BỘ NHỚ

8.1 CACHE MEMORY

Cache memory là bộ nhớ **nhỏ, rất nhanh**, được đặt giữa CPU và bộ nhớ chính. Nó lưu bản sao của dữ liệu / lệnh **được dùng thường xuyên**.

MỤC ĐÍCH

- ✓ Giảm thời gian truy cập bộ nhớ trung bình
- ✓ Thu hẹp khoảng cách tốc độ giữa CPU và bộ nhớ chính
- ✓ Cải thiện hiệu năng tổng thể của hệ thống

CÁCH CACHE HOẠT ĐỘNG

- CPU yêu cầu dữ liệu.
- Nếu dữ liệu có trong cache → **Cache Hit** (truy cập nhanh).
- Nếu dữ liệu không có trong cache → **Cache Miss**.
 - Block cần thiết được nạp từ bộ nhớ chính.
 - Block được lưu vào cache (thay thế một block nào đó nếu cần).
 - Sau đó dữ liệu được gửi đến CPU.

HIỆU NĂNG CACHE

$$\text{Hit Ratio (h)} = \frac{\text{Số lần Hit}}{\text{Tổng số lần truy cập}}$$

$$\text{Miss Ratio (m)} = 1 - h$$

Average Memory Access Time (AMAT)

$$= (h \times \text{Hit Time}) + (m \times \text{Miss Penalty})$$

Hit Time → Thời gian truy cập cache

Miss Penalty → Thời gian nạp dữ liệu từ bộ nhớ chính

TỔ CHỨC CACHE

Cache được chia thành các **block** (line). Mỗi block chứa bản sao của một block trong bộ nhớ chính.



CÁC LOẠI CACHE

- L1 Cache:** Nhỏ nhất & nhanh nhất, tích hợp ngay trong CPU.
- L2 Cache:** Lớn hơn, chậm hơn một chút, có thể nằm trong / ngoài chip.
- L3 Cache:** Lớn nhất, chậm hơn L1 & L2, được chia sẻ giữa các core.

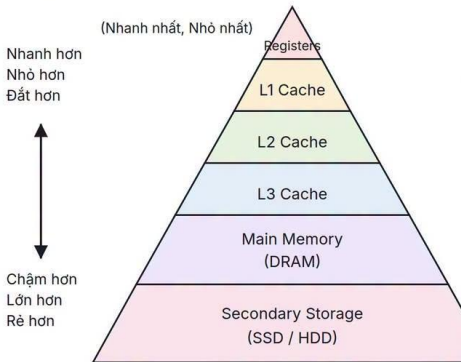
KỸ THUẬT ÁNH XẠ CACHE

Kỹ thuật	Mô tả	Ví dụ
1. Direct Mapping	Một block bộ nhớ chỉ có thể được đặt vào một cache line cụ thể.	Block i → Line (i mod n)
2. Fully Associative	Một block bộ nhớ có thể được đặt vào bất kỳ cache line nào.	Block i → Bất kỳ line nào
3. Set Associative	Cache được chia thành các set. Block có thể được đặt vào bất kỳ line nào trong set của nó.	Block i → Bất kỳ line nào trong set (i mod s)

Tag: Xác định block bộ nhớ nào đang được lưu.
Index: Xác định cache line / set.
Offset: Xác định byte / word bên trong block.

8.2 MEMORY HIERARCHY

Memory hierarchy (phân cấp bộ nhớ) là cấu trúc nhiều cấp gồm các bộ nhớ, mỗi cấp có dung lượng, tốc độ và chi phí khác nhau.



CÁC CẤP CỦA MEMORY HIERARCHY

Cấp	Ví dụ	Tốc độ	Dung lượng	Chi phí / bit	Quản lý bởi
1. Registers	CPU Registers	Rất cao	Rất nhỏ	Rất cao	Compiler / CPU
2. L1 Cache	On-chip	Rất cao	Nhỏ	Cao	Hardware
3. L2 Cache	On-chip / Off-chip	Cao	Trung bình	Trung bình	Hardware
4. L3 Cache	Off-chip	Trung bình	Lớn	Thấp	Hardware
5. Main Memory	DRAM	Thấp	Lớn	Rất thấp	OS / Hardware
6. Secondary Storage	SSD / HDD	Rất thấp	Rất lớn	Cực thấp	OS

NGUYÊN TẮC CỦA MEMORY HIERARCHY

Giữ dữ liệu được dùng nhiều nhất ở các cấp trên (nhanh hơn).
Chuyển dữ liệu ít dùng hơn xuống các cấp dưới (chậm hơn).



9. TỔ CHỨC INPUT / OUTPUT



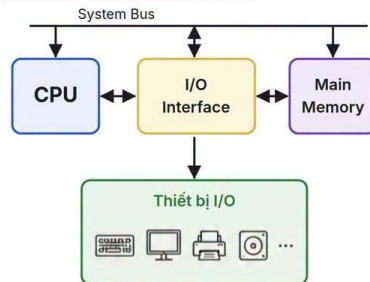
Tổ chức I/O đề cập đến cách máy tính giao tiếp với các thiết bị bên ngoài. Nó bao gồm các thiết bị, giao diện (interface), phương thức truyền dữ liệu và cơ chế điều khiển cho phép thực hiện các thao tác vào / ra.

1. THIẾT BỊ I/O

Các thiết bị I/O cho phép máy tính tương tác với thế giới bên ngoài.

Thiết bị đầu vào	Thiết bị đầu ra
Bàn phím	Màn hình
Chuột	Máy in
Máy quét	Loa
Micro	Máy vẽ (Plotter)
Cảm biến, v.v.	Cơ cấu chấp hành (Actuator), v.v.

2. TỔ CHỨC I/O CƠ BẢN



I/O Interface
Đóng vai trò trung gian giữa CPU / bộ nhớ và các thiết bị I/O. Nó quản lý dữ liệu, điều khiển và định thời.

3. I/O INTERFACE (CONTROLLER)

- Interface (hay controller) quản lý việc giao tiếp giữa CPU / bộ nhớ và các thiết bị I/O.
- Nó bao gồm:
 - **Data Register**: Lưu dữ liệu cần gửi hoặc nhận.
 - **Status Register**: Chứa thông tin trạng thái của thiết bị.
 - **Control Register**: Chứa các tín hiệu điều khiển và lệnh.
- Nó xử lý định thời, handshaking và phát hiện lỗi.

5. HANDSHAKING (ĐỒNG BỘ HÓA)



Handshaking đảm bảo sự phối hợp đúng đắn giữa CPU và thiết bị I/O trong quá trình truyền dữ liệu.

6. CÁC THAO TÁC I/O

- CPU phát lệnh I/O bằng cách ghi vào I/O interface.
- I/O interface kiểm tra trạng thái thiết bị.
- Dữ liệu được truyền bằng một trong các phương thức truyền.
- Trạng thái được cập nhật và CPU được thông báo (nếu cần).
- Dữ liệu đã sẵn sàng trong bộ nhớ hoặc thiết bị.

8. MEMORY-MAPPED VS I/O-MAPPED I/O

Đặc điểm	Memory-Mapped I/O	I/O-Mapped (Isolated) I/O
Không gian địa chỉ	Dùng chung không gian địa chỉ bộ nhớ	Dùng không gian địa chỉ I/O riêng
Lệnh sử dụng	Lệnh đọc / ghi bộ nhớ	Lệnh I/O đặc biệt (IN / OUT)
Đường địa chỉ	Dùng toàn bộ các đường địa chỉ	Thường dùng ít đường địa chỉ hơn
Phần cứng	Đơn giản	Phức tạp hơn một chút

4. PHƯƠNG THỨC TRUYỀN DỮ LIỆU

Phương thức	Mô tả	Ưu điểm	Nhược điểm
Programmed I/O	CPU phát lệnh I/O rồi chờ (poll) cho đến khi thao tác hoàn tất.	Đơn giản, dễ triển khai	CPU rảnh rỗi trong lúc I/O
Interrupt Driven I/O	CPU khởi động I/O rồi tiếp tục thực thi các tác vụ khác. Thiết bị I/O ngắt CPU khi hoàn tất.	Tận dụng CPU tốt hơn	Chi phí xử lý ngắt (overhead)
Direct Memory Access (DMA)	Dữ liệu được truyền trực tiếp giữa thiết bị I/O và bộ nhớ mà không cần CPU can thiệp.	Tốc độ cao, CPU can thiệp tối thiểu	Phần cứng phức tạp hơn

7. CẤU TRÚC I/O INTERFACE (VÍ DỤ)

Base + 0	→	Data Register	←	Dữ liệu đến / từ thiết bị
Base + 1	→	Status Register	←	Trạng thái thiết bị
Base + 2	→	Control Register	←	Lệnh điều khiển
Base + 3	→	Command Register	←	Start / Stop / Reset, v.v.

CPU truy cập các register này bằng memory-mapped I/O hoặc các lệnh I/O riêng (isolated I/O).

9. ĐIỂM CHÍNH

- Thiết bị I/O cho phép giao tiếp giữa máy tính và thế giới bên ngoài.
- I/O interface điều khiển và quản lý mọi thao tác I/O.
- Truyền dữ liệu có thể là Programmed, Interrupt-driven hoặc DMA.
- Handshaking đảm bảo trao đổi dữ liệu tin cậy.
- Memory-mapped I/O dùng chung không gian địa chỉ với bộ nhớ, còn I/O-mapped dùng không gian riêng.

10.

SYSTEM BUS & KIẾN TRÚC BUS



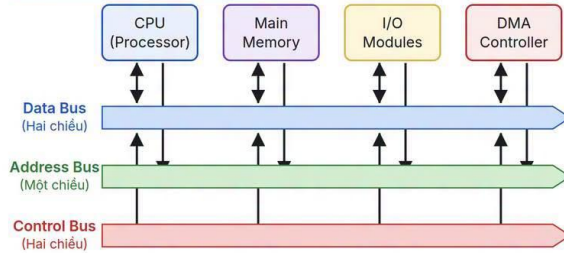
System bus là tập hợp các đường truyền thông tin dùng để chuyển dữ liệu, địa chỉ và tín hiệu điều khiển giữa CPU, bộ nhớ và các thiết bị I/O.

Nó đóng vai trò là đường truyền thông tin bên trong máy tính.

1. SYSTEM BUS LÀ GÌ?

- Bus là đường truyền thông tin dùng chung.
- Nó kết nối mọi thành phần chính của hệ thống máy tính.
- Nó mang dữ liệu, địa chỉ và các tín hiệu điều khiển.
- Nó quyết định các thành phần giao tiếp nhanh và hiệu quả đến đâu.

2. KIẾN TRÚC BUS CƠ BẢN



3. CÁC LOẠI BUS

Loại bus	Chiều	Mục đích	Tín hiệu ví dụ
Data Bus	Hai chiều	Truyền dữ liệu thực tế giữa CPU, bộ nhớ và I/O.	D0 - D7, D8 - D31
Address Bus	Một chiều	Mang địa chỉ từ CPU đến bộ nhớ hoặc thiết bị I/O.	A0 - A15, A0 - A31
Control Bus	Hai chiều	Mang tín hiệu điều khiển và định thời.	Read, Write, Memory Select, I/O Select, Interrupt, Clock, Reset

4. CHI TIẾT CẤU TRÚC BUS

- Độ rộng Data Bus (n bit)**: Lượng dữ liệu truyền được tại một thời điểm.
 - Bus rộng hơn → nhiều dữ liệu hơn mỗi xung nhịp → hiệu năng cao hơn.
- Độ rộng Address Bus (m bit)**: Quyết định số địa chỉ bộ nhớ tối đa.
 - Bộ nhớ tối đa có thể đánh địa chỉ = 2^m vị trí.
- Control Bus**: Điều phối mọi hoạt động trên bus.
 - Bao gồm các tín hiệu điều khiển, định thời và trạng thái.

Ví dụ:

Nếu address bus có 20 đường → CPU có thể đánh địa chỉ 2^{20} vị trí = 1 MB (khi mỗi vị trí là 1 byte).



5. HOẠT ĐỘNG CỦA BUS (VÍ DỤ: ĐỌC BỘ NHỚ)

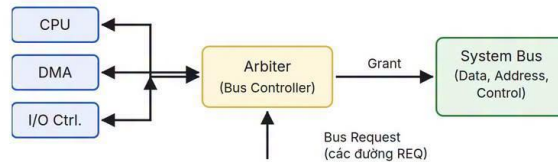
	CPU	Bus	Memory
Bước 1	Đặt địa chỉ lên Address Bus	Địa chỉ →	Nhận địa chỉ
Bước 2	Gửi tín hiệu Read	Tín hiệu Read →	Chuẩn bị dữ liệu
Bước 3	Chờ	Dữ liệu sẵn sàng ←	Đặt dữ liệu lên Data Bus
Bước 4	Đọc dữ liệu từ Data Bus	Dữ liệu ←	Giữ dữ liệu
Bước 5	Bỏ tín hiệu Read	Read không hoạt động →	Sẵn sàng cho thao tác tiếp theo

7. ĐẶC TÍNH CỦA BUS

Đặc tính	Mô tả
Độ rộng bus	Số đường dữ liệu / địa chỉ. Bus rộng hơn → nhiều dữ liệu hơn, nhiều địa chỉ hơn.
Tốc độ bus	Tốc độ dữ liệu có thể được truyền đi.
Băng thông bus	Tổng khả năng truyền dữ liệu (Bandwidth = Width x Speed).
Khả năng mở rộng	Khả năng hỗ trợ thêm thiết bị.
Định thời (Timing)	Đồng bộ hóa tín hiệu để hoạt động đúng.

6. PHÂN XỬ BUS (BUS ARBITRATION)

- Khi nhiều thiết bị (VD: CPU, DMA, I/O controller) cần dùng bus, một cơ chế phân xử (arbitration) được sử dụng.
- Nó quyết định thiết bị nào giành được quyền điều khiển bus.



Arbiter dùng chính sách ưu tiên hoặc công bằng để tránh xung đột.

8. CÁC KIẾN TRÚC BUS PHỔ BIẾN

Kiến trúc	Mô tả
Single Bus	Một bus dùng cho cả truyền dữ liệu bộ nhớ và I/O.
Separate Bus	Bus riêng cho bộ nhớ và bus riêng cho I/O.
Hierarchical Bus	Bus tốc độ cao gần CPU, các bus chậm hơn cho thiết bị I/O.
Point-to-Point	Liên kết trực tiếp giữa các thành phần (các hệ thống hiện đại).

11. PIPELINING



Pipelining là kỹ thuật dùng để tăng thông lượng lệnh (instruction throughput) của CPU. Nó chia việc thực thi lệnh thành một chuỗi các giai đoạn (stage). Trong khi một lệnh đang ở một giai đoạn, các lệnh khác có thể đang ở các giai đoạn khác cùng lúc. **Giống như một dây chuyền lắp ráp!**

1. Ý TƯỞNG CƠ BẢN

- Trong thực thi không pipeline, mỗi lệnh đi qua tất cả các giai đoạn, lần lượt từng lệnh một.
- Trong thực thi có pipeline, nhiều lệnh chồng lấp nhau trong quá trình thực thi.
- Điều này làm tăng throughput (số lệnh hoàn thành trên một đơn vị thời gian).

THUẬT NGỮ CHÍNH

- Throughput**: Số lệnh hoàn thành trên một đơn vị thời gian.
- Latency**: Thời gian để hoàn thành một lệnh.
- Pipeline**: Một chuỗi các giai đoạn, mỗi giai đoạn thực hiện một phần của việc thực thi lệnh.

2. CÁC GIAI ĐOẠN CỦA INSTRUCTION PIPELINE (VÍ DỤ 5 GIAI ĐOẠN)



- IF : Nạp lệnh từ bộ nhớ dùng PC
ID : Giải mã lệnh và đọc toán hạng từ các register
EX : Thực hiện phép toán ALU / tính địa chỉ
MEM : Truy cập bộ nhớ cho các lệnh load / store
WB : Ghi kết quả trở lại register

Các bộ xử lý khác nhau có thể có nhiều hoặc ít giai đoạn hơn, nhưng ý tưởng là như nhau.

3. THỰC THI KHÔNG PIPELINE VS CÓ PIPELINE

(a) Thực thi không Pipeline

Mỗi lệnh hoàn thành trước khi lệnh tiếp theo bắt đầu.

Thời gian → (chu kỳ)

Lệnh	1	2	3	4	5	6	7	8	9	10
I1	IF	ID	EX	MEM	WB					
I2						IF	ID	EX	MEM	WB
I3										...

(b) Thực thi có Pipeline

Nhiều lệnh ở các giai đoạn khác nhau cùng một lúc.

Lệnh	1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	EX	MEM	WB		
I3			IF	ID	EX	MEM	WB	
I4				IF	ID	EX	MEM	WB

Khi pipeline đã đầy (sau n chu kỳ với n giai đoạn), **mỗi chu kỳ hoàn thành một lệnh!**

6. CÁC PIPELINE HAZARD

Loại hazard	Mô tả	Ví dụ	Giải pháp
Structural Hazard	Hai giai đoạn cần cùng một phần cứng tại cùng thời điểm.	Một bộ nhớ dùng chung cho IF và MEM.	Dùng bộ nhớ lệnh và bộ nhớ dữ liệu riêng.
Data Hazard	Lệnh phụ thuộc vào kết quả của lệnh trước.	ADD R1, R2, R3 SUB R4, R1, R5	Forwarding, Stalling (Bubble)
Control Hazard	Lệnh tiếp theo phụ thuộc vào kết quả rẽ nhánh / nhảy.	BEQ R1, R2, Label	Branch Prediction, Stalling, Flush

8. ĐIỂM QUAN TRỌNG

- Pipelining tăng throughput, không làm giảm latency của từng lệnh.
- Speedup lý tưởng bằng số giai đoạn (k) của pipeline.
- Hazard là nguyên nhân chính khiến pipeline không đạt speedup lý tưởng.
- Forwarding và branch prediction giúp giảm số lần stall.

4. CẢI THIỆN HIỆU NĂNG

Gọi,

k = số giai đoạn trong pipeline

t = thời gian của mỗi giai đoạn

n = số lệnh

• Thời gian không Pipeline = $n \times k \times t$

• Thời gian có Pipeline = $(k - 1 + n) \times t$

$$\text{Speedup} = \frac{n \times k}{k - 1 + n} \approx k \text{ (với n lớn)}$$

5. ƯU ĐIỂM VÀ NHƯỢC ĐIỂM

Ưu điểm	Nhược điểm
✓ Tăng thông lượng lệnh (hiệu năng tốt hơn)	✗ Thiết kế bộ xử lý phức tạp hơn
✓ Tận dụng CPU cao hơn	✗ Pipeline hazard làm giảm hiệu năng
✓ Sử dụng phần cứng hiệu quả	✗ Cần nhiều register hơn (pipeline register giữa các giai đoạn)

7. VÍ DỤ VỀ PIPELINE HAZARD

Data Hazard (RAW - Read After Write)

I1: ADD R1, R2, R3
I2: SUB R4, R1, R5 ↔ I2 cần kết quả R1 từ I1

Giải pháp:

- Chuyển tiếp (forward) kết quả từ giai đoạn EX/MEM của I1 sang giai đoạn EX của I2.
- Nếu không được, chèn một stall (bubble).

TỔNG KẾT



Pipelining cho phép CPU thực thi nhiều lệnh chồng lấp nhau như một dây chuyền lắp ráp, tăng đáng kể thông lượng - nhưng cần xử lý tốt các hazard để đạt hiệu quả cao nhất.

12. RISC VS CISC



RISC và CISC là hai cách tiếp cận khác nhau trong thiết kế tập lệnh (instruction set) của kiến trúc máy tính. Chúng đánh đổi giữa sự đơn giản và hiệu năng.

Đặc điểm	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Tên đầy đủ	Reduced Instruction Set Computer	Complex Instruction Set Computer
Triết lý thiết kế	Dùng các lệnh đơn giản, mỗi lệnh mất 1 chu kỳ xung nhịp. Tập trung vào tốc độ và hiệu quả.	Dùng các lệnh phức tạp, có thể mất nhiều chu kỳ xung nhịp. Tập trung vào việc giảm kích thước mã.
Tập lệnh	Nhỏ và đơn giản	Lớn và phức tạp
Độ dài lệnh	Cố định (VD: 32 bit)	Thay đổi
Addressing Modes	Ít (Hạn chế)	Nhiều (Phong phú)
Số lượng lệnh	Ít hơn	Nhiều hơn
Thời gian thực thi	Hầu hết lệnh thực thi trong 1 chu kỳ xung nhịp	Lệnh có thể mất nhiều chu kỳ xung nhịp
Control Unit	Đơn giản	Phức tạp (microprogrammed)
Độ phức tạp phần cứng	Thấp hơn	Cao hơn
Registers	Nhiều register đa dụng	Ít register đa dụng hơn
Truy cập bộ nhớ	Kiến trúc Load/Store (Chỉ LOAD và STORE truy cập bộ nhớ)	Nhiều lệnh có thể truy cập bộ nhớ (VD: ADD [memory], JUMP [memory])
Compiler	Cần nhiều lệnh hơn, nhưng dễ tối ưu hơn	Cần ít lệnh hơn, nhưng khó tối ưu hơn
Kích thước mã	Lớn hơn	Nhỏ hơn
Hiệu năng	Cao hơn (nhiều lệnh hơn mỗi giây)	Thấp hơn (ít lệnh hơn mỗi giây)
Ví dụ	ARM, MIPS, SPARC, RISC-V, PowerPC	x86, x86-64, 68000, VAX

RISC vs CISC: Ví dụ thực thi

Biểu thức: $A = B + C * D$

RISC (Lệnh đơn giản)

```
1. LOAD R1, B ; R1 = B
2. LOAD R2, C ; R2 = C
3. LOAD R3, D ; R3 = D
4. MUL R2, R2, R3 ; R2 = C * D
5. ADD R1, R1, R2 ; R1 = B + (C*D)
6. STORE R1, A ; A = R1
```

Tổng: 6 lệnh

CISC (Lệnh phức tạp)

```
1. MOV AX, B
2. MUL C, D ; AX = C * D
3. ADD AX, B ; AX = B + (C*D)
4. MOV A, AX
```

Tổng: 4 lệnh

CISC dùng ít lệnh hơn để thực hiện cùng một tác vụ, nhưng mỗi lệnh có thể mất nhiều hơn một chu kỳ xung nhịp.

Ưu điểm của RISC

- Lệnh đơn giản cho phép thực thi nhanh hơn.
- Dễ thiết kế và triển khai hơn.
- Hiệu năng cao hơn và pipelining tốt hơn.
- Nhiều register hơn → ít truy cập bộ nhớ hơn.
- Tiêu thụ điện năng thấp hơn (nói chung).

Nhược điểm của RISC

- Cần nhiều lệnh hơn cho các tác vụ phức tạp.
- Kích thước mã lớn hơn.
- Cần nhiều băng thông bộ nhớ hơn.

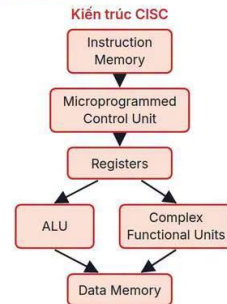
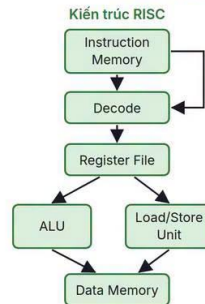
Ưu điểm của CISC

- Ít lệnh hơn cho các tác vụ phức tạp.
- Kích thước mã nhỏ hơn.
- Tốt hơn cho hệ thống hạn chế bộ nhớ.
- Addressing mode phức tạp giúp giảm thao tác bộ nhớ.

Nhược điểm của CISC

- Lệnh phức tạp mất nhiều chu kỳ xung nhịp hơn.
- Khó thiết kế và triển khai hơn.
- Hiệu năng thấp hơn so với RISC.
- Tiêu thụ điện năng cao hơn.

RISC vs CISC: So sánh cấu trúc



RISC vs CISC: Tóm tắt chính

- ✓ RISC đơn giản, nhanh và hiệu quả.
- ✓ CISC phức tạp, gọn và tiết kiệm mã.
- ✓ Bộ xử lý RISC được dùng trong các hệ thống di động, nhúng và hiệu năng cao hiện đại.
- ✓ Bộ xử lý CISC thường được dùng trong máy tính để bàn và máy chủ đa dụng (họ x86).



Xu hướng hiện đại

Các bộ xử lý hiện đại (như x86) bên trong sử dụng các kỹ thuật kiểu RISC:

- Pipelining
- Superscalar execution
- Out-of-order execution
- Register renaming



Vì vậy, các lệnh CISC được giải mã thành các micro-operation đơn giản hơn và được thực thi trong một lõi kiểu RISC.



TỔNG KẾT

RISC tập trung vào làm những việc đơn giản thật nhanh, còn CISC tập trung vào làm những việc phức tạp với ít bước hơn. Kiến trúc tốt nhất là kiến trúc đạt hiệu năng tối đa với chi phí và điện năng tối thiểu.



13.

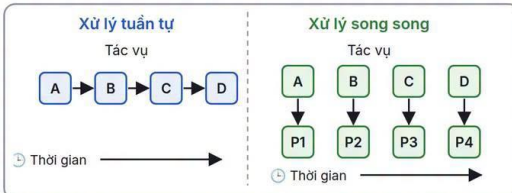
XỬ LÝ SONG SONG (Parallel Processing)



Parallel Processing (xử lý song song) là kỹ thuật sử dụng **nhiều đơn vị xử lý (core/CPU)** cùng lúc để giải quyết một bài toán **nhANH HƠN**. Bài toán được chia thành các phần nhỏ hơn, được thực thi đồng thời rồi kết hợp lại để có kết quả cuối cùng.

1. Ý TƯỞNG CƠ BẢN

- Trong một bộ xử lý đơn, các tác vụ được thực thi lần lượt (tuần tự).
- Trong xử lý song song, các tác vụ được thực thi cùng lúc trên các bộ xử lý khác nhau.



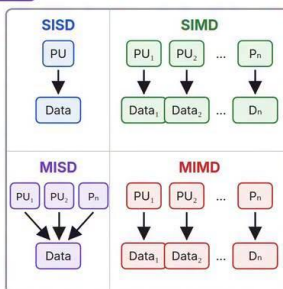
2. CÁC LOẠI SONG SONG

- Tính song song có thể đạt được ở nhiều mức khác nhau.

Loại	Mô tả	Ví dụ
Bit-Level Parallelism	Thao tác trên nhiều bit dữ liệu cùng lúc.	Bộ cộng 32-bit cộng 32 bit song song.
Instruction-Level Parallelism (ILP)	Nhiều lệnh được thực thi chồng lấn nhau.	Pipelining, thực thi Superscalar.
Data-Level Parallelism (DLP)	Cùng một thao tác trên các phần tử dữ liệu khác nhau đồng thời.	Vector processor, lệnh SIMD.
Task-Level Parallelism (TLP)	Các tác vụ khác nhau được thực thi đồng thời.	CPU đa lõi, Multi-threading, hệ thống phân tán.

3. CÁC KIẾN TRÚC XỬ LÝ SONG SONG

- SISD** - Single Instruction, Single Data (Hệ thống bộ xử lý đơn truyền thống)
- SIMD** - Single Instruction, Multiple Data (Cùng một lệnh trên nhiều dữ liệu)
- MISD** - Multiple Instruction, Single Data (Hiếm khi được dùng)
- MIMD** - Multiple Instruction, Multiple Data (Tổng quát nhất và được dùng rộng rãi nhất)



PU → Processing Unit (có thể là core / processor)

6. VÍ DỤ: CỘNG VECTOR

Tuần tự
Cộng hai mảng A và B có kích thước 8.
 $C[i] = A[i] + B[i]$
for $i = 0$ to 7
 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$
(Từng phần tử một)

Song song (4 bộ xử lý)
Chia công việc cho 4 bộ xử lý.
 $P1 \rightarrow 0,1$ $P2 \rightarrow 2,3$ $P3 \rightarrow 4,5$ $P4 \rightarrow 6,7$
Tất cả được tính cùng một lúc

4. LỢI ÍCH

- ✓ Thực thi nhanh hơn và thông lượng cao hơn
- ✓ Tận dụng tài nguyên tốt hơn
- ✓ Có thể giải các bài toán lớn và phức tạp
- ✓ Cải thiện độ tin cậy (khả năng chịu lỗi)
- ✓ Hiệu năng có thể mở rộng (thêm bộ xử lý)

5. THÁCH THỨC

- ✗ Chi phí giao tiếp và đồng bộ hóa
- ✗ Thiết kế phần cứng và phần mềm phức tạp hơn
- ✗ Cân bằng tải khó khăn
- ✗ Chương trình song song khó viết và gỡ lỗi hơn
- ✗ Không phải bài toán nào cũng có thể song song hóa

7. ỨNG DỤNG



8. SO SÁNH: ĐƠN LỖI VS ĐA LỖI (SONG SONG)

Đặc điểm	Đơn lõi (Tuần tự)	Đa lõi / Xử lý song song
Thực thi	Một tác vụ tại một thời điểm	Nhiều tác vụ cùng lúc
Hiệu năng	Thấp hơn với các tác vụ lớn	Cao hơn với tác vụ lớn / có thể song song hóa
Tận dụng tài nguyên	Có thể chưa tận dụng hết	Tận dụng tài nguyên CPU tốt hơn
Hiệu quả năng lượng	Tốt hơn cho các tác vụ nhỏ	Hiệu quả hơn cho khối lượng công việc lớn
Độ phức tạp	Đơn giản	Phức tạp hơn (đồng bộ hóa)

9. ĐIỂM CHÍNH

- 🔥 Xử lý song song dùng nhiều bộ xử lý để làm việc đồng thời trên các phần của một tác vụ.
- 🔥 Nó làm tăng tốc độ và hiệu quả.
- 🔥 Đạt được thông qua các loại song song khác nhau.
- 🔥 MIMD là kiến trúc tổng quát và thực tế nhất.
- 🔥 Được dùng rộng rãi trong CPU đa lõi, GPU và điện toán đám mây.

14.

BÀI TẬP TÍNH TOÁN & ĐỀ THI QUAN TRỌNG



Phần này bao gồm các bài toán tính toán quan trọng nhất thường gặp trong các kỳ thi về **Kiến trúc Máy tính**.
Hãy học các công thức, hiểu phương pháp và luyện tập thường xuyên!

1. TÍNH TOÁN HIỆU NĂNG

(a) CPU Time

CPU Time = Instruction Count $\times$ CPI $\times$ Clock Cycle Time
Clock Cycle Time = $1 / \text{Clock Rate}$

(b) MIPS (Million Instructions Per Second)

$$\text{MIPS} = \frac{\text{Instruction Count}}{\text{CPU Time} \times 10^6}$$

(c) Ví dụ

Một chương trình có 10^9 lệnh.
CPI = 2, Clock Rate = 2 GHz.
Tìm CPU Time và MIPS.

Lời giải:

$$\text{CPU Time} = 10^9 \times 2 \times \frac{1}{(2 \times 10^9)} = 1 \text{ giây}$$

$$\text{MIPS} = \frac{10^9}{1 \times 10^6} = 1000 \text{ MIPS}$$

4. BÀI TOÁN CACHE

(a) Số cache line

$$\text{Số line} = \frac{\text{Cache Size}}{\text{Block Size}}$$

(b) Ví dụ

Cache size = 32 KB, Block size = 64 Byte
Tìm số cache line.

$$\text{Số line} = 32 \times 1024 / 64 = 512 \text{ line}$$

(c) Phân tích địa chỉ

Cache Size	Block Size	Address Bits	Tag Bits	Index Bits	Offset Bits
32 KB	64 B	32	32 - (9 + 6)	9	6

$$\text{Offset bit} = \log_2(\text{Block Size}) = \log_2(64) = 6$$

$$\text{Index bit} = \log_2(\text{Số line}) = \log_2(512) = 9$$

$$\text{Tag bit} = 32 - (6 + 9) = 17 \text{ bit}$$

7. BẢNG THÔNG BUS

$$\text{Bandwidth} = \frac{\text{Bus Width (bit)}}{8} \times \frac{\text{Bus Frequency (Hz)}}{10^6} \text{ (MB/s)}$$

Ví dụ:

Bus width = 64 bit, Bus frequency = 100 MHz
Bandwidth = $(64/8) \times 100 = 800 \text{ MB/s}$

10. CÂU HỎI NGẮN QUAN TRỌNG (CÓ ĐÁP ÁN)

STT	Câu hỏi	Trả lời ngắn
1	MIPS là gì?	Million Instructions Per Second. Đo số triệu lệnh mà CPU thực thi được trong một giây.
2	CPI là gì?	Cycles Per Instruction. Số chu kỳ xung nhịp trung bình cần để thực thi một lệnh.
3	Ưu điểm của cache là gì?	Giảm thời gian truy cập bộ nhớ trung bình bằng cách lưu dữ liệu thường dùng gần CPU hơn.
4	Miss penalty là gì?	Thời gian bổ sung để nạp dữ liệu từ bộ nhớ chính khi xảy ra cache miss.

2. ĐỊNH LUẬT AMDAHL (SPEEDUP)

$$\text{Speedup (S)} = \frac{1}{(1 - P) + P/N}$$

Trong đó,

P = Phần chương trình có thể được cải thiện

N = Số bộ xử lý

(1 - P) = Phần vẫn chạy tuần tự

Ví dụ:

80% của chương trình có thể song song hóa.

Tìm speedup khi dùng 4 bộ xử lý.

$$P = 0.8, N = 4$$

$$S = 1 / [(1 - 0.8) + 0.8/4]$$

$$= 1 / [0.2 + 0.2]$$

$$= 1 / 0.4 = 2.5$$

5. DIRECT MAPPED CACHE

(a) Số thứ tự Line

$$\text{Line Number} = (\text{Block Address}) \bmod (\text{Số line})$$

(b) Ví dụ

Một block bộ nhớ có địa chỉ 120 cần được lưu vào cache có 16 line. Block size = 4 word.

$$\text{Block Address} = 120 / 4 = 30$$

$$\text{Line Number} = 30 \bmod 16 = 14$$

(c) Hit hay Miss?

Nếu tag khớp và valid bit = 1 $\rightarrow$ Hit

Ngược lại $\rightarrow$ Miss

8. THỜI GIAN CHU KỲ LỆNH

$$\text{Cycle Time} = \frac{1}{\text{Clock Rate}}$$

Ví dụ:

Clock Rate = 2 GHz

$$\text{Cycle Time} = 1 / (2 \times 10^9) = 0.5 \text{ ns}$$

3. THỜI GIAN TRUY CẬP BỘ NHỚ (CÓ CACHE)

$$\text{Average Memory Access Time (AMAT)} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty}$$

Trong đó,

Hit Time = Thời gian truy cập cache (khi hit)

Miss Penalty = Thời gian truy cập bộ nhớ chính

Ví dụ:

Hit Time = 2 ns, Miss Rate = 5%

Miss Penalty = 100 ns

$$\text{AMAT} = 2 + 0.05 \times 100$$

$$= 2 + 5 = 7 \text{ ns}$$

$$\text{Miss Rate} = \frac{\text{Số lần Miss}}{\text{Tổng số lần truy cập bộ nhớ}}$$

6. VIRTUAL MEMORY

(a) Page Number và Offset

$$\text{Page Number} = \frac{\text{Virtual Address}}{\text{Page Size}}$$

$$\text{Offset} = \text{Virtual Address} \bmod \text{Page Size}$$

(b) Ví dụ

Page size = 4 KB (2^{12}), Virtual Address = 0x12345

$$\text{Page Number} = 0x12345 / 0x1000 = 0x12$$

$$\text{Offset} = 0x12345 \bmod 0x1000 = 0x345$$

(c) Định dạng Page Table Entry (Ví dụ)

Valid	Protection	Reference	Frame Number
1 bit	3 bit	1 bit	m bit

9. CPI TRUNG BÌNH

$$\text{Average CPI} = \frac{\sum (\text{CPI}_i \times \text{Instruction}_i)}{\text{Tổng số lệnh}}$$

Ví dụ:

Loại A: 2 CPI, 40% số lệnh

Loại B: 1 CPI, 60% số lệnh

$$\text{Average CPI} = (2 \times 0.4) + (1 \times 0.6) = 1.4$$

CÔNG THỨC THAM KHẢO NHANH

- CPU Time = IC $\times$ CPI $\times$ Cycle Time
- MIPS = IC / (CPU Time $\times 10^6$)
- Speedup (Amdahl) = $1 / ((1 - P) + P/N)$
- AMAT = Hit Time + Miss Rate $\times$ Miss Penalty
- #Lines = Cache Size / Block Size
- Bandwidth = (Bus Width/8) $\times$ (Bus Frequency/10⁶)
- Cycle Time = $1 / \text{Clock Rate}$
- Average CPI = $\sum (\text{CPI}_i \times \text{Instruction}_i) / \text{Tổng số lệnh}$

15. CÂU HỎI PHÒNG VẤN + CHEAT SHEET ÔN TẬP

Ôn thông minh
Điểm thật cao!



Luyện tập các câu hỏi phỏng vấn này để xây dựng kiến thức vững chắc và sự tự tin. Hãy giữ cheat sheet bên mình để ôn nhanh trước các kỳ thi và buổi phỏng vấn!

A. CÂU HỎI PHÒNG VẤN QUAN TRỌNG (kèm ý chính)

1 Sự khác biệt giữa RISC và CISC là gì?

- RISC:** Lệnh đơn giản, độ dài cố định, kiến trúc load/store, nhiều register hơn, dễ pipelining hơn.
- CISC:** Lệnh phức tạp, độ dài thay đổi, thao tác memory-to-memory, ít register hơn.

2 Pipelining là gì? Ưu điểm và hazard của nó?

- Pipelining chia việc thực thi lệnh thành các giai đoạn và chống lấp chúng để tăng throughput.
- Ưu điểm:** Hiệu năng cao hơn, tận dụng CPU tốt hơn.
- Hazard:** Structural, Data, Control.

3 Cache memory là gì? Tại sao cần dùng nó?

- Cache là bộ nhớ nhỏ, tốc độ cao nằm giữa CPU và bộ nhớ chính.
- Nó lưu dữ liệu / lệnh thường dùng để giảm thời gian truy cập bộ nhớ trung bình và cải thiện hiệu năng.

4 Vai trò của bus trong hệ thống máy tính là gì?

- Bus là đường truyền thông tin chuyển dữ liệu, địa chỉ và tín hiệu điều khiển giữa CPU, bộ nhớ và các thiết bị I/O.
- Các loại:** Data Bus, Address Bus, Control Bus.

5 Sự khác biệt giữa SRAM và DRAM là gì?

- SRAM:** Nhanh hơn, không cần làm tươi (refresh), đắt, dùng cho cache.
- DRAM:** Chậm hơn, cần làm tươi định kỳ, rẻ hơn, dùng cho bộ nhớ chính.

6 Virtual memory là gì? Nó hoạt động như thế nào?

- Virtual memory cho phép thực thi các chương trình lớn hơn bộ nhớ chính bằng cách dùng bộ nhớ phụ như một phần mở rộng.
- Dùng paging / segmentation và page table để dịch địa chỉ.

7 Instruction cycle là gì?

- Fetch → Decode → Execute → Memory Access → Write Back.
- Chu trình này lặp lại cho mọi lệnh.

8 Addressing mode là gì? Kể tên một vài loại.

- Chúng chỉ định cách xác định vị trí của toán hạng.
- Các loại:** Immediate, Register, Direct, Indirect, Register Indirect, Indexed, Base + Displacement.

9 Xử lý song song là gì? Được dùng ở đâu?

- Dùng nhiều bộ xử lý / lõi cùng lúc để giải quyết một bài toán.
- Dùng trong CPU (đa lõi), GPU, siêu máy tính, AI/ML, mô phỏng khoa học.

10 Làm thế nào để cải thiện hiệu năng cache?

- Tăng kích thước cache, giảm miss rate, dùng chính sách thay thế tốt hơn (LRU), tăng độ liên kết (associativity).



Mẹo hay: Khi trả lời, hãy đưa ví dụ thực tế (CPU, mobile, server), vẽ sơ đồ nếu cần, và giữ câu trả lời ngắn gọn, có cấu trúc.



B. CHEAT SHEET ÔN TẬP NHANH

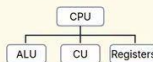
1. Kiến trúc Máy tính

- Nghiên cứu cấu trúc, hành vi và cách triển khai của hệ thống máy tính.
- Trọng tâm: CPU, Memory, I/O và sự tương tác giữa chúng.



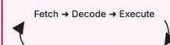
2. Tổ chức CPU

- ALU:** Thực hiện phép toán số học & logic.
- CU:** Điều khiển mọi hoạt động.
- Registers:** Lưu dữ liệu, lệnh, địa chỉ.



3. Instruction Cycle

- Fetch (Lấy lệnh)
- Decode (Giải mã)
- Execute (Thực hiện)
- Memory Access
- Write Back



4. Định dạng lệnh

Các trường: Opcode | Addressing Mode | Operand / Address

Loại: Data Transfer, Arithmetic, Logical, Branch, I/O, System.



5. Addressing Modes

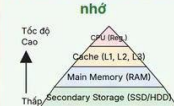
- Immediate
- Register
- Direct
- Indirect
- Register Indirect
- Indexed
- Base + Displacement

6. Tổ chức bộ nhớ

- Bộ nhớ chính (RAM)
- ROM / Bộ nhớ bất biến
- Tổ chức phụ thuộc vào độ dài word, không gian địa chỉ, độ rộng dữ liệu.
- Loại: SRAM, DRAM, ROM, Flash, EEPROM.



7. Cache & phân cấp bộ nhớ



- Cấp cao hơn = nhanh hơn & nhỏ hơn
- Cấp thấp hơn = chậm hơn & lớn hơn
- Mục tiêu: Giảm thiểu thời gian truy cập bộ nhớ trung bình

8. Tổ chức I/O

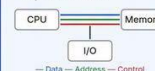
- Thiết bị I/O kết nối qua interface / controller.
- Loại: Programmed I/O, Interrupt-driven I/O, DMA.
- Truyền dữ liệu: CPU ↔ I/O Module ↔ Device.



9. System Bus

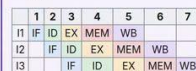
Ba bus chính:

- Data Bus:** Mang dữ liệu
- Address Bus:** Mang địa chỉ
- Control Bus:** Mang tín hiệu điều khiển



10. Pipelining

- Chia việc thực thi lệnh thành các giai đoạn.
- Chống lấp các giai đoạn để cải thiện throughput.
- 5 giai đoạn: IF, ID, EX, MEM, WB.



11. Xử lý song song

- Nhiều bộ xử lý / lõi làm việc đồng thời.
- Loại: SISD, SIMD, MISD, MIMD.
- Ứng dụng: CPU đa lõi, GPU, AI/ML, điện toán đám mây.

12. RISC vs CISC

Đặc điểm	RISC	CISC
Lệnh	Đơn giản	Phức tạp
Độ dài	Cố định	Thay đổi
Addressing	Ít	Nhiều
Register	Nhiều	Ít

13. Công thức hiệu năng

- CPU Time = IC × CPI × Clock Cycle Time
- MIPS = IC / (CPU Time × 10⁶)
- Speedup (Amdahl) = 1 / ((1 - P) + P/N)
- AMAT = Hit Time + Miss Rate × Miss Penalty

14. Các đơn vị thường gặp

- ALU - Arithmetic & Logic
- CU - Control Unit
- Registers - Bộ nhớ nhanh
- Cache - Bộ nhớ đệm tốc độ cao
- DMA - Direct Memory Access

15. Điểm quan trọng

- ✓ Hit rate cache cao hơn = hiệu năng tốt hơn.
- ✓ Pipelining cải thiện throughput, không phải latency.
- ✓ Nhiều register hơn = ít truy cập bộ nhớ hơn.
- ✓ Bus width quyết định lượng dữ liệu truyền mỗi chu kỳ.